

Практическая работа 14

ИСПОЛНИТЕЛИ, ВИРТУАЛЬНЫЕ ПОТОКИ И ГОНКИ ДАННЫХ

Цель работы

Показать воспроизводимую потерю обновления, исправить её атомарной операцией, дождаться всех Future и корректно завершить исполнители виртуальных потоков.

Описание программы

Демонстрация сравнивает два неатомарных read-modify-write с AtomicInteger, параллельно обрабатывает снимок трёх планов, передаёт исходную ошибку и отменяет ожидающую задачу.

Общие сведения

ExecutorService управляет жизненным циклом задач, Future представляет результат или отказ, а виртуальный поток уменьшает стоимость блокирующего ожидания. Эти средства не делают count++ атомарным и не разрешают делить JDBC-соединение.

Функциональное назначение и ограничения

Сценарий нужен для безопасного распараллеливания независимых диагностик с явным ожиданием всех результатов и контролируемым закрытием executor.

Планы читаются до параллельного участка. В задачи передаётся List.copyOf, но не Repository, Connection и не JavaFX-узлы.

Логическая структура

Барьер заставляет две задачи прочитать одно значение до записи, поэтому unsafe-счётчик получает 1. AtomicInteger даёт 2. Future собираются в порядке входа; ExecutionException сохраняет cause, а закрытие подтверждается отсутствием активных задач.

Используемые технические средства

Java 25, ExecutorService, виртуальные потоки, Future, AtomicInteger, CountDownLatch, JUnit 5 и Gradle 9.1.

Входные и выходные данные

Входные данные. Неизменяемый список планов, параллельные задания, учебная ошибка ID 102 и команда отмены ожидающей задачи.

Выходные данные. unsafe=1 и atomic=2, три результата, сохранённая причина ошибки и подтверждение interrupt/cancel/closed.

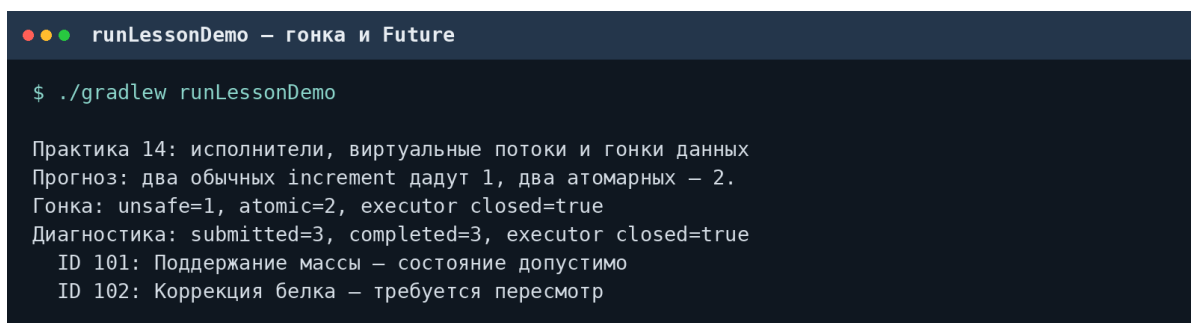
Контрольные сценарии

Компонент	Ответственность	Проверка
ExecutorService	Запуск и закрытие задач	runLessonDemo
Future	Результат, ошибка, отмена	JUnit
Immutable snapshot	Граница общих данных	Архитектурный тест

Ход работы

Проект собран и проверен по README и протоколу проверки. Ниже приведены фактические результаты запусков и снимки текущей реализации; учебные изображения преподавателя не использовались.

1. Выполнена демонстрация с барьером. Неатомарный счётчик потерял одно обновление, AtomicInteger сохранил оба, а три Future завершились в порядке исходного снимка.



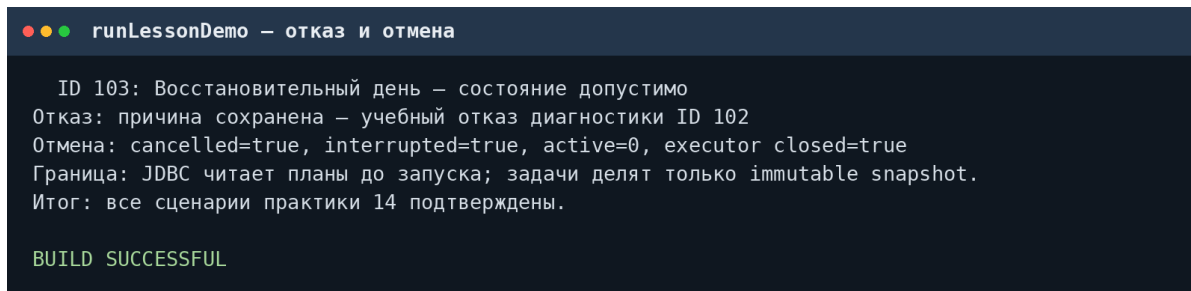
```
runLessonDemo - гонка и Future

$ ./gradlew runLessonDemo

Практика 14: исполнители, виртуальные потоки и гонки данных
Прогноз: два обычных increment дадут 1, два атомарных – 2.
Гонка: unsafe=1, atomic=2, executor closed=true
Диагностика: submitted=3, completed=3, executor closed=true
ID 101: Поддержание массы – состояние допустимо
ID 102: Коррекция белка – требуется пересмотр
```

Рисунок 1 – Гонка, атомарное исправление и ожидание результатов

2. Проверены отрицательные ветви. Исходная причина диагностики ID 102 сохранена, ожидающая задача получила interrupt, активных заданий после закрытия не осталось.



```
runLessonDemo - отказ и отмена

ID 103: Восстановительный день – состояние допустимо
Отказ: причина сохранена – учебный отказ диагностики ID 102
Отмена: cancelled=true, interrupted=true, active=0, executor closed=true
Граница: JDBC читает планы до запуска; задачи делят только immutable snapshot.
Итог: все сценарии практики 14 подтверждены.

BUILD SUCCESSFUL
```

Рисунок 2 – Передача причины, interrupt и закрытие executor

Вывод

Параллельный сценарий ожидает все задачи и закрывает исполнители. Гонка воспроизводится детерминированно, AtomicInteger устраняет потерю обновления, а ошибка и отмена остаются диагностируемыми.

Выполнил: студент группы 090302-ИСТа-023	Карионов В.В.
Проверил: преподаватель	Елкин Н.С.